Mining Data Streams (Part 1)

Data Streams

- **In many data mining situations, we do not know the entire data set in advance**

- **Stream Management** is important when the input rate is controlled **externally:**
  - Google queries
  - Twitter or Facebook status updates
- We can think of the **data** as **infinite** and **non-stationary** (the distribution changes over time)

The Stream Model

- Input **elements** enter at a rapid rate, at one or more input ports (i.e., **streams**)
  - **We call elements of the stream tuples**
- **The system cannot store the entire stream accessibly**
- **Q: How do you make critical calculations about the stream using a limited amount of (secondary) memory?**

1 THINK LIKE GOOGLE

Q: Google receives billions of search queries every day. Why can't Google store all queries and process them at midnight?

Follow-up

- What are the limitations?
- Is the problem storage, computation, or both?
- What would happen if processing is delayed by 24 hours?

Key Takeaway

- Too much data to store
- Need real-time insights
- Stream processing solves this

2 STREAM OR DATABASE?

Decide which is more suitable and why.

Application	Database	Stream	Why?
Student Management System	✓		Finite data
Google Search		✓	High volume, real-time
Netflix Recommendations		✓	Continuous events
ECG Monitoring		✓	Real-time, time critical
WhatsApp Messages		✓	Continuous flow
ATM Transactions		✓	Real-time fraud detection
Library Management	✓		Static / occasional updates
Smart Traffic Lights		✓	Sensor data in real-time

3 FINITE OR INFINITE?

Which of the following are infinite streams (∞) and which are finite datasets (✓)?

Infinite Streams (∞)

- YouTube comments
- Temperature sensor
- IPL live scores
- Heart rate monitor
- WhatsApp messages

Finite Datasets (✓)

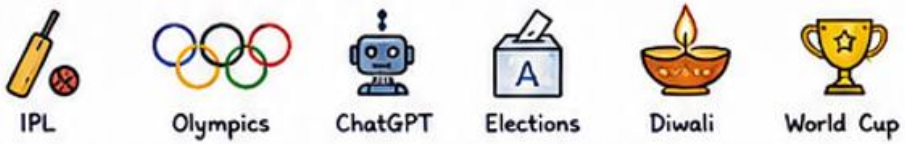
- University student records
- Passport database
- Election results
- Product catalog
- Library books list

Follow-up: Can a finite dataset become a stream? Give an example.

Example: A one-time sale table → becomes a stream during live sale days.

4 WHY IS THE DATA NON-STATIONARY?

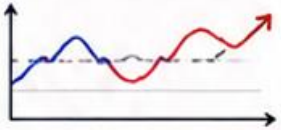
Real-world events change over time.



Think & Discuss

- Why is yesterday's popular search different from today's?
- What challenge does this create for machine learning models?
- How can models adapt to these changes?

Key Idea: Data is infinite and non-stationary (the distribution changes over time).



Side note: SGD is a Streaming Alg.

- **Stochastic Gradient Descent (SGD) is an example of a stream algorithm**
- **In Machine Learning we call this: Online Learning**
  - Allows for modeling problems where we have a continuous stream of data
  - We want an algorithm to learn from it and slowly adapt to the changes in data
- **Idea: Do slow updates to the model**
  - **SGD** (SVM, Perceptron) makes small updates
  - **So:** First train the classifier on training data.
  - **Then:** For every example from the stream, we slightly update the model (using small learning rate)

5 DESIGN A STREAM ELEMENT (TUPLE)

What would one stream tuple look like?

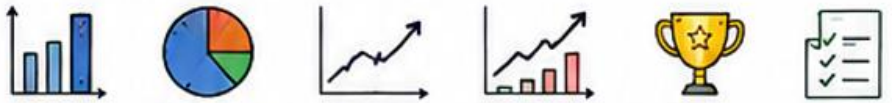
- Uber → (UserID, DriverID, Location, Time)
- Netflix → (UserID, MovieID, WatchTime, Rating)
- Instagram → (UserID, PostID, Action, Time)
- Smart Hospital → (PatientID, HeartRate, Time, DeviceID)
- Smart Agriculture → (SensorID, Temp, Humidity, Time)

Choose attributes that are important for analysis.

6 WHAT WOULD YOU STORE?

Your server has RAM = 2 GB but receives 100 GB of data every hour.

What information would you keep?



In streams, we keep summaries, not raw data!

7 IS SGD REALLY A STREAMING ALGORITHM?

Q: Why?

SGD (Stochastic Gradient Descent)

- After every new training example, the model is updated slightly.
- It does not need the complete dataset in memory.
- Works with one example (or a small batch) at a time.

→

Batch Gradient Descent	vs.	SGD
Needs entire dataset		Uses one example at a time
Computationally expensive		Fast & memory efficient
Update after full pass		Update after every example

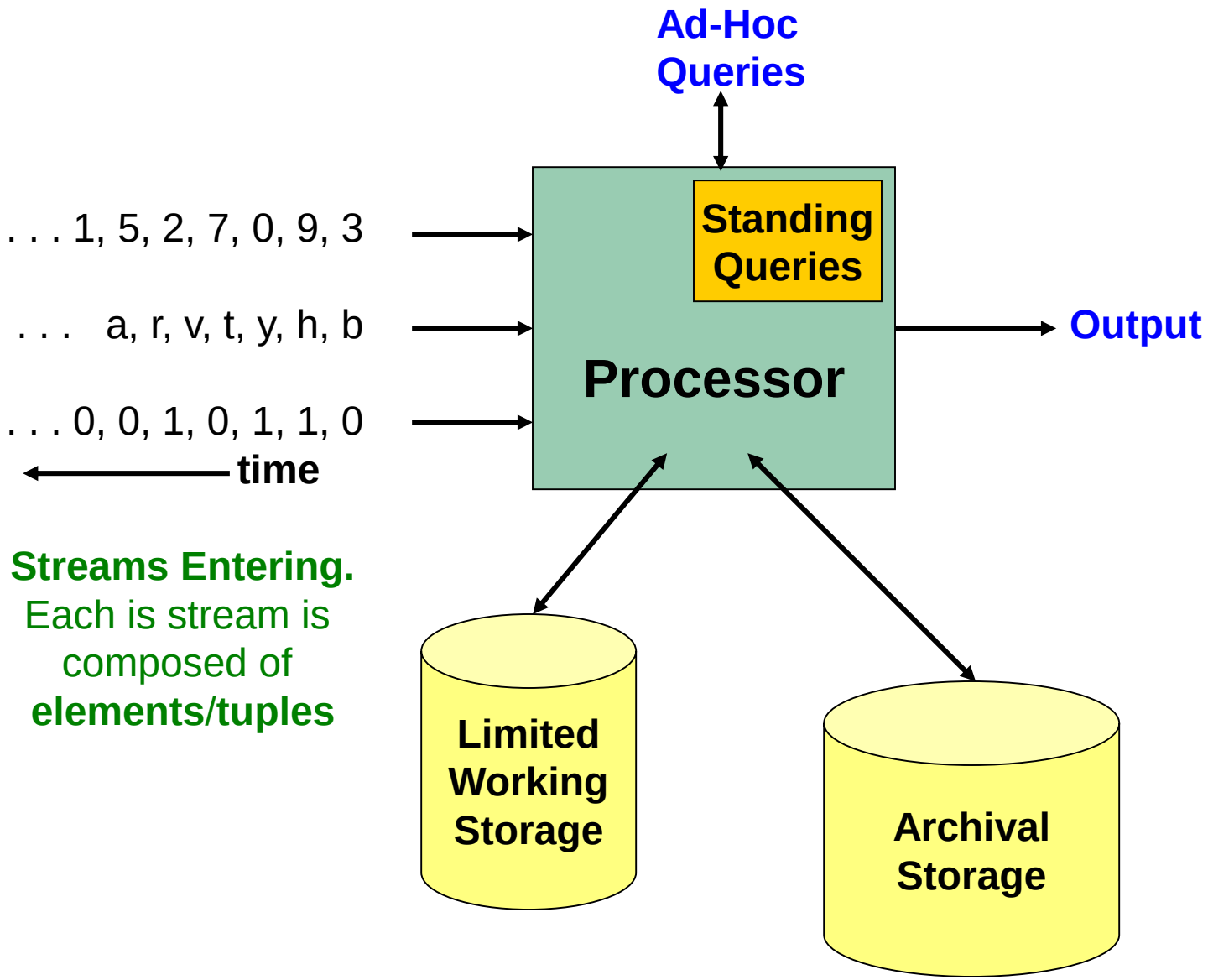
Therefore, SGD is a streaming (online) algorithm.

8 ONLINE vs OFFLINE LEARNING

Batch (Offline) Learning	Online Learning
Trains on entire dataset at once	Learns incrementally from stream
High memory requirement	Low memory requirement
Model updated after full training	Model updated continuously
Slow, not suitable for real-time	Fast, suitable for real-time
Example: Linear Regression (on full data)	Example: SGD, Perceptron

Netflix, YouTube, Google → prefer Online Learning!

General Stream Processing Model



Streams Entering. Each is stream is composed of **elements/tuples**

GENERAL STREAM PROCESSING MODEL

1 Incoming Streams

Multiple data streams enter the system continuously.

- Google Search Queries
- Twitter / X Updates
- Sensor Readings (IoT Devices)
- Bank Transactions
- … More Streams

Each element is a tuple

(UserID, Query, Time)

Elements (Tuples)

Ad-Hoc Queries (asked on demand)

Standing Queries (always active)

Processor

Analyzes each arriving element and produces results in real time.

Output

Limited Working Storage (RAM)

Archival Storage (Old Data)

4 Output (Real-time Results)

- Dashboards
- Alerts & Notifications
- Reports
- Recommendations



5 Limited Working Storage

- Small memory (e.g., RAM)
- Cannot store entire stream
- Stores only summaries (not raw data)



6 Archival Storage

- Stores old / historical data
- Used for future analysis, auditing, machine learning training, etc.



2 Stream Elements (Tuples)

Examples

UserID	Search	Time
Alice	AI	10:00
Bob	Weather	10:01
Charlie	Flights	10:02
…	…	…

Each row = one element (tuple)

3 Standing vs Ad-hoc Queries

Standing Queries	Ad-hoc Queries
Run continuously	Executed only when requested
Monitor every incoming tuple	Used for analysis and reporting
Used for alerts, real-time detection	

7 Real-World Examples

- Google → Trending Searches
- Netflix → Recommendations
- Smart Health → Alerts
- Banking → Fraud Detection
- IoT → Sensor Monitoring

8 Key Takeaways

- Data arrives continuously as streams (tuples).
- Processor analyzes in real time using limited memory.
- Standing queries → always running
- Ad-hoc queries → on demand
- Old data → archived for future use
- Output → valuable real-time info

Problems on Data Streams

- **Types of queries one wants on answer on a data stream:**
  - **Sampling data from a stream**
    - Construct a random sample
  - **Queries over sliding windows**
    - Number of items of type ***x*** in the last ***k*** elements of the stream

Problems on Data Streams

- **Types of queries one wants on answer on a data stream:**

- **Filtering a data stream**
  - Select elements with property ***x*** from the stream
  - **Counting distinct elements**
    - Number of distinct elements in the last ***k*** elements of the stream
  - **Estimating moments**
    - Estimate avg./std. dev. of last ***k*** elements
  - **Finding frequent elements**

Applications (1)

- **Mining query streams**
  - Google wants to know what queries are more frequent today than yesterday
- **Mining click streams**
  - Yahoo wants to know which of its pages are getting an unusual number of hits in the past hour
- **Mining social network news feeds**
  - E.g., look for trending topics on Twitter, Facebook

Applications (2)

- **Sensor Networks**
  - Many sensors feeding into a central controller
- **Telephone call records**
  - Data feeds into customer bills as well as settlements between telephone companies
- **IP packets monitored at a switch**
  - Gather information for optimal routing
  - Detect denial-of-service attacks

Sampling from a Data Stream: Sampling a fixed proportion

As the stream grows the sample also gets bigger

Sampling from a Data Stream

- Since **we can not store the entire stream**, one obvious approach is to store a **sample**
- **Two different problems:**
  - **(1)** Sample a **fixed proportion** of elements in the stream (say 1 in 10)
  - **(2)** Maintain a **random sample of fixed size** over a potentially infinite stream
    - At any "time" ***k*** we would like a random sample of ***s*** elements
      - **What is the property of the sample we want to maintain?** For all time steps ***k***, each of ***k*** elements seen so far has equal prob. of being sampled

Sampling a Fixed Proportion

- **Problem 1: Sampling fixed proportion**
- **Scenario:** Search engine query stream
  - **Stream of tuples:** (user, query, time)
  - **Answer questions such as:** **How often did a user run the same query in a single days**
  - Have space to store **1/10th** of query stream
- **Naïve solution:**
  - Generate a random integer in **[0..9]** for each query
  - Store the query if the integer is **0**, otherwise discard

Problem with Naïve Approach

- **Simple question: What fraction of queries by an average search engine user are duplicates?**
  - Suppose each user issues ***x*** queries once and ***d*** queries twice (total of ***x*+2*d*** queries)
    - **Correct answer: *d*/(*x*+*d*)**
  - **Proposed solution: We keep 10% of the queries**
    - Sample will contain ***x*/10** of the singleton queries and **2*d*/10** of the duplicate queries at least once
    - But only ***d*/100** pairs of duplicates
      - **d/100 = 1/10 · 1/10 · d**
    - Of ***d*** "duplicates" ***18d/100*** appear exactly once
      - **18d/100 = ((1/10 · 9/10)+(9/10 · 1/10)) · d**
  - **So the sample-based answer is** $\frac{\frac{d}{100}}{\frac{x}{10}+\frac{d}{100}+\frac{18d}{100}} = \frac{d}{10x+19d}$

1 What is the Question?

Google wants to know:

"What fraction of queries by an average search engine user are duplicate queries?"

User	Queries
Alice	AI
Bob	Weather
Charlie	AI, AI

Only Charlie searched the same query twice.

We need the % of all queries that are duplicates.

2 Assume a Simple Model

Each user issues:

- x unique queries (searched once)
- d queries repeated twice

Example: $x = 8,\ d = 2$

User searches:

- 8 unique: AI, Weather, Movies, IPL, Flights, Hotels, News, Maps
- Repeats 2 queries: AI, Movies

3 Total Number of Queries

Every duplicated query appears twice.

Duplicate entries $= 2d$

Total queries $= x + 2d$

Example:

$x = 8,\ d = 2$

Total $= 8 + 2(2) = 12$

(8 unique + 4 duplicate entries)

4 Correct Answer (What we want)

We count distinct query types:

- x singleton query types
- d duplicated query types

Total query types = $x + d$

True fraction of duplicate queries = $\frac{d}{x+d}$

This is the value Google wants to estimate.

5 Naïve Sampling (10%)

Google stores only 10% of all queries.

1. Generate a random number (0–9) for each query.
2. Store the query only if the number is 0.

Each query has 1/10 chance of being selected.



→ Seems fair... but here is the problem!

6 What Happens to Singleton Queries?

Suppose there are x singleton queries.

Each has $\frac{1}{10}$ chance of being selected.

Expected number stored = $x \times \frac{1}{10} = \frac{x}{10}$

Singletons stored in sample = $\frac{x}{10}$

7 What Happens to Duplicate Queries?

Consider one duplicate query pair: (AI, AI)

Both copies are sampled independently (p = 1/10).

First copy	Second copy	Probability	Result
Keep	Keep	1/10 × 1/10 = 1/100	Appears twice
Keep	Discard	1/10 × 9/10 = 9/100	Appears once
Discard	Keep	9/10 × 1/10 = 9/100	Appears once
Discard	Discard	9/10 × 9/10 = 81/100	Lost

Only 1/100 of the pairs remain as duplicates.

18/100 appear exactly once (look like singletons now!).

8 Expected Counts in Sample

For d duplicate pairs:

- True duplicate pairs left = $d/100$
- Broken duplicates (appear once) = $18d/100$

Total items in sample:

- Original singletons: : $x/10$
- Broken duplicates: : $18d/100$
- True duplicate pairs: : $d/100$

9 Sample-Based Estimate (Derivation)

$$\text{Estimated duplicate fraction} = \frac{\frac{d}{100}}{\frac{x}{10} + \frac{d}{100} + \frac{18d}{100}}$$

Multiply by 100:

$$\text{Estimated fraction} = \frac{d}{10x + 19d}$$

10 Why is the Naïve Approach Wrong?

- ✗ Duplicate queries are sampled independently.
- ✗ Often only one copy is selected, so a duplicate looks like a single query.
- ✗ Relationships between related records are broken.
- ✗ This underestimates the true fraction of duplicates (biased result).



Key Takeaway

Randomly sampling individual queries destroys duplicate relationships and produces a biased result. Better approach → sample users (or another key), so that all queries from a selected user are kept together.

Solution: Sample Users

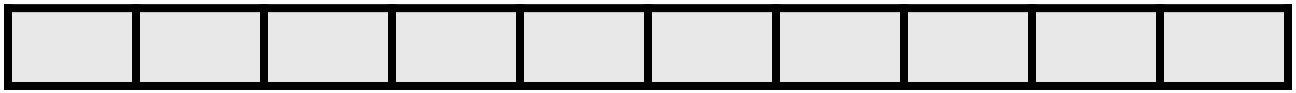
Solution:

- Pick **1/10th** of **users** and take all their searches in the sample
- Use a hash function that hashes the user name or user id uniformly into 10 buckets

Generalized Solution

- **Stream of tuples with keys:**
  - Key is some subset of each tuple's components
    - e.g., tuple is (user, search, time); key is **user**
  - Choice of key depends on application

- **To get a sample of *a/b* fraction of the stream:**
  - Hash each tuple's key uniformly into ***b*** buckets
  - Pick the tuple if its hash value is at most ***a***



Hash table with **b** buckets, pick the tuple if its hash value is at most **a.**
How to generate a 30% sample?
Hash into b=10 buckets, take the tuple if it hashes to one of the first 3 buckets

HASH-BASED SAMPLING IN DATA STREAMS

Instead of randomly deciding for every tuple, we consistently sample tuples based on a key using hashing.

1 WHAT IS A TUPLE?

The search engine receives the following stream.

User	Search	Time
Alice	AI	10:00
Bob	Weather	10:01
Alice	NLP	10:02
Charlie	IPL	10:03
Bob	Flights	10:04
David	AI	10:05

Each row is called a tuple.

A tuple has multiple attributes (columns):

(User, Search, Time)

2 WHAT IS THE KEY?

Key is some subset of each tuple's components.

Example tuple: (Alice, AI, 10:00)

Possible keys:

- User
- Search Query
- Time
- Location
- Device ID

} Choice depends on application.

If you want to study user behaviour
→ Key = User

If you want to study popular searches
→ Key = Search Query

3 HASH INTO b BUCKETS

Suppose $b = 10$. Create 10 logical buckets.

Buckets:	0	1	2	3	4	5	6	7	8	9

Now every key is passed through a hash function.

User	Hash(User)	Bucket
Alice	132	2
Bob	450	0
Charlie	821	1
David	637	7
Emma	913	3
Frank	204	4

Hash function automatically distributes users among buckets (uniformly).

④ SUPPOSE WE NEED A 30% SAMPLE



Choose $a = 3$

$b = 10$

Therefore,

$$\frac{a}{b} = \frac{3}{10} = 30\%$$

Keep buckets 0, 1, 2

Discard others.

0	1	2	3	4	6	7	8	9

KEEP DISCARD

WHY DOES THIS WORK?

A good hash function distributes keys uniformly.

Suppose there are 100,000 users.

Bucket	0	1	2	3	4	5	6	7	8	9
Users	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000

If we keep buckets 0, 1, 2 → Stored users = 30,000 = 30%

NUMERICAL EXAMPLE

Incoming Stream (User → Bucket)

User	Bucket
Alice	2
Bob	8
Charlie	1
David	6
Emma	0
Frank	4
Grace	9
Henry	3
John	1
Lily	7

Keep Buckets: 0, 1, 2

User	Bucket	Keep?
Alice	2	✓
Bob	8	✗
Charlie	1	✓
David	6	✗
Emma	0	✓
Frank	4	✗
Grace	9	✗
Henry	3	✗
John	1	✓
Lily	7	✗

FINAL SAMPLE

- ✓ Alice
- ✓ Charlie
- ✓ Emma
- ✓ John

We kept 4 out of 10 ≈ 40%

With very large datasets (millions of users), the sample approaches the desired 30%.

5 OTHER EXAMPLES (DIFFERENT SAMPLE FRACTIONS)

20% SAMPLE

Need 20%

Choose a = 1, b = 5

because $\frac{1}{5}$ = 20%

Buckets: 0 | 1 | 2 | 3 | 4

Keep only bucket 0

Expected result: 20%

60% SAMPLE

Need 60%

Choose a = 6, b = 10

Buckets: 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Keep 0, 1, 2, 3, 4, 5

Discard 6, 7, 8, 9

Expected result: 60%

75% SAMPLE

Need 75%

Choose a = 3, b = 4

Buckets: 0 | 1 | 2 | 3

Keep 0, 1, 2

Discard 3

Result: 3/4 = 75%

GENERAL RULE

To get a sample of a/b fraction:

1. Hash each key uniformly into b buckets.
2. Keep the tuple if its bucket number is < a (i.e., in buckets 0 to a-1).

Sample Fraction = $\frac{a}{b}$

6 WHY USE HASHING INSTEAD OF RANDOM NUMBERS?

Suppose Alice searches 100 times.

RANDOM SAMPLING (10%)

Each query is sampled independently.

Q1	Q2	Q3	...	Q100
✗	✓	✗	...	✓

Sometimes selected, sometimes not.

Inconsistent and breaks user behaviour.

vs.

HASHING ON USER ID

Alice always hashes to the same bucket (say bucket 2).

If bucket 2 is selected: ✓ ✓ ✓ ... ✓ ✓

All 100 searches kept

If bucket 2 is not selected: ✗ ✗ ✗ ... ✗

None of Alice's searches kept

KEY TAKEAWAY

Hashing provides consistency. It keeps all records of some keys and excludes others completely, preserving relationships and providing unbiased, scalable sampling in data streams.

Sampling from a Data Stream: Sampling a fixed-size sample

As the stream grows, the sample is of fixed size



Maintaining a fixed-size sample

- **Problem 2: Fixed-size sample**
- **Suppose we need to maintain a random sample *S* of size exactly *s* tuples**
  - E.g., main memory size constraint
- **Why?** Don't know length of stream in advance
- **Suppose at time *n* we have seen *n* items**
  - **Each item is in the sample *S* with equal prob. *s/n***

How to think about the problem: say s = 2

Stream: a x c y z k c d e g…

At **n= 5,** each of the first 5 tuples is included in the sample **S** with equal prob.

At **n= 7,** each of the first 7 tuples is included in the sample **S** with equal prob.

Impractical solution would be to store all the *n* tuples seen so far and out of them pick *s* at random

Solution: Fixed Size Sample

- **Algorithm (a.k.a. Reservoir Sampling)**
  - Store all the first ***s*** elements of the stream to ***S***
  - Suppose we have seen ***n-1*** elements, and now the ***nth*** element arrives (***n*** > ***s***)
    - With probability ***s/n***, keep the ***nth*** element, else discard it
    - If we picked the ***nth*** element, then it replaces one of the ***s*** elements in the sample ***S***, picked uniformly at random
- **Claim:** This algorithm maintains a sample ***S*** with the desired property:
  - After ***n*** elements, the sample contains each element seen so far with probability ***s/n***



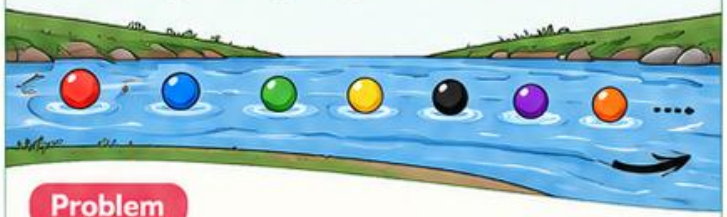
RESERVOIR SAMPLING

Keep a small, fair sample from an endless stream

Fixed size sample
Equal chance for all!

THE SITUATION

Items (balls) arrive one by one in a stream that may be very long or infinite.



Problem

- ? We don't know how many items will come.
- ? The stream may never end.
- ✗ We cannot store all items (memory is limited).

Goal

Maintain exactly s items (sample/reservoir) such that **EVERY** item seen so far has **equal chance** of being in it.

THE IDEA (RIVER ANALOGY)

You stand by a river of balls. Your bag can hold only $s = 3$ balls.

1. First s balls: Just take them.



Bag: [🔴 🔵 🟢]

2. Next ball arrives (🟡). Maybe take it. If yes, replace one randomly.



Example: Replace 🔵

Bag: [🔴 🟡 🟢]

3. Next ball arrives (⚫). Maybe take it. If yes, replace one randomly.



Example: Replace 🟢

Bag: [🔴 🟡 ⚫]

4. Continue forever... Always keep exactly $s = 3$ balls in the bag.



Always 3 balls in the bag!

In the end, every ball that ever passed by had an equal chance of being in the bag.

REAL-LIFE EXAMPLES

Google Search

Billions of searches arrive every day. Keep a random sample of 1000 searches for analysis.

Twitter / X

Millions of tweets per second. Keep a sample to detect trends without storing every tweet.

IoT Sensors

Sensors send readings continuously. Keep a sample of readings for monitoring and anomaly detection.

Stock Market

Prices update every second. Keep a sample of ticks to analyze patterns and volatility.

Lucky Draw

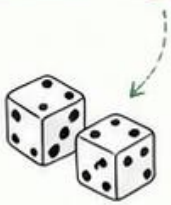
People register one by one. Keep only a few names in the box so that everyone has an equal chance.

HOW IT WORKS (in simple steps)

1. **Initialization**
   Put the first s items into the sample (reservoir).



2. **For the i-th new item** (where $i > s$)
   - Generate a random integer r between 1 and i.
   - If $r \leq s$, replace the item at position r in the sample with the new item.
   - Else, discard the new item.



3. **Repeat Step 2 for every new incoming item.**

At any time, the sample has size s and every seen item has equal probability of being in it.

WHY IT IS FAIR (INTUITION)

When a new item arrives:

It may be discarded



Old items stay.

Or it is chosen and replaces one old item randomly



Old items still have a chance.



Result

After many steps, every item—old or new—gets the same chance $\frac{s}{n}$ of being in the sample when n items have been seen.

FORMAL DEFINITION

Reservoir Sampling is an algorithm used to maintain a fixed-size random sample from a data stream of unknown or infinite length, ensuring that every element seen so far has an equal probability of being included in the sample.

WHY DO WE NEED IT?

- Uses constant memory (only s items)
- Processes data one item at a time
- Does not store the entire stream
- Produces an unbiased random sample

ONE-LINE INTUITION

Reservoir Sampling is like carrying a small bucket while standing beside an endless river—you cannot collect every drop of water, so you continuously replace old drops in a way that ensures every drop has an equal chance of remaining in the bucket.

Key Takeaway: Reservoir Sampling lets us take a fair, **random sample** of fixed size from any stream, no matter how large or how long it is, using very little memory.

Proof: By Induction

- **We prove this by induction:**
  - Assume that after ***n*** elements, the sample contains each element seen so far with probability ***s/n***
  - We need to show that after seeing element ***n+1*** the sample maintains the property
    - Sample contains each element seen so far with probability ***s/(n+1)***
- **Base case:**
  - After we see **n=s** elements the sample **S** has the desired property
    - Each out of **n=s** elements is in the sample with probability ***s/s = 1***

Proof: By Induction

- **Inductive hypothesis:** After ***n*** elements, the sample ***S*** contains each element seen so far with prob. ***s/n***
- **Now element *n+1* arrives**
- **Inductive step:** For elements already in ***S***, probability that the algorithm keeps it in ***S*** is:

$$\left(1-\frac{s}{n+1}\right)+\left(\frac{s}{n+1}\right)\left(\frac{s-1}{s}\right)=\frac{n}{n+1}$$

Element **n+1** discarded

Element **n+1** not discarded

Element in the sample not picked

- So, at time ***n***, tuples in ***S*** were there with prob. **s/n**
- Time ***n→n+1***, tuple stayed in ***S*** with prob. **n/(n+1)**
- So prob. tuple is in ***S*** at time ***n+1*** = $\frac{s}{n}\cdot\frac{n}{n+1}=\frac{s}{n+1}$

PROOF BY INDUCTION (RESERVOIR SAMPLING)

Goal: After seeing $n+1$ elements, every element is in the sample S with probability $\frac{s}{n+1}$.

s = sample size
n = number of elements seen so far
S = current sample (reservoir)

1 INDUCTIVE HYPOTHESIS

Assume after seeing n elements, each element is in S with probability $\frac{s}{n}$.

Example

Let $s = 2$ and $n = 4$.

Stream seen:

A B C D

(4 elements)

By the hypothesis, each element has probability $\frac{2}{4} = \frac{1}{2}$ of being in S.

Suppose current sample $S = [B, D]$

2 NOW ELEMENT $n+1$ ARRIVES

A new element arrives (the $(n+1)$-th element).

Continuing the example:

A B C D E

↑ New element

Now $n+1 = 5$. We want every element A, B, C, D, E to have probability $\frac{2}{5}$ of being in the sample.

3 HOW DOES AN OLD ELEMENT STAY?

Consider any old element (say B) that is currently in S. For B to remain in the sample after E arrives, two situations can happen.

Case 1: New element is discarded

Reservoir Sampling chooses the new element with probability $\frac{s}{n+1}$.

So, probability E is discarded is

$$1 - \frac{s}{n+1} = 1 - \frac{2}{5} = \frac{3}{5}$$

If E is discarded, B stays.

Case 2: E is selected, but B is NOT replaced

E is selected with probability $\frac{s}{n+1} = \frac{2}{5}$.

Now E replaces one of the s positions.

Probability that B is NOT replaced (i.e., the other position is replaced) is $\frac{s-1}{s} = \frac{1}{2}$.

So, probability of this case $= \frac{2}{5} \times \frac{1}{2} = \frac{1}{5}$

B stays in this case because E replaces someone else.

4 TOTAL PROBABILITY THAT B SURVIVES

Add the two cases:

Case 1 $\frac{3}{5}$ + Case 2 $\frac{1}{5}$ = Total $\frac{4}{5}$ = $\frac{n}{n+1}$ = $\frac{4}{5}$

So, the probability that an old element survives is $\frac{n}{n+1}$.

5 OLD ELEMENT WAS ALREADY IN THE SAMPLE

Before E arrived, B was already in the sample with probability $\frac{s}{n} = \frac{2}{4}$.

Now, probability B is in S after $n+1$ elements
= (old prob.) × (survives)

$$\frac{s}{n} \times \frac{n}{n+1} = \frac{2}{4} \times \frac{4}{5} = \frac{2}{5}$$

Perfect! B now has probability $\frac{2}{5}$.

6 WHAT ABOUT THE NEW ELEMENT E?

E is selected with probability $\frac{s}{n+1} = \frac{2}{5}$ and if selected, it enters the sample.

Therefore, probability $(E \in S)$ = $\frac{2}{5}$

7 FINAL RESULT

After processing $n+1 = 5$ elements, every element has the same probability of being in the sample. $\frac{2}{5}$

COMPLETE EXAMPLE ($s = 2$)

Step	Stream Seen	Sample S (one possible)	Prob. of each seen element)
$n = 2$	A, B	[A, B]	2/2 = 1
$n = 3$	A, B, C	[A, C]	2/3
$n = 4$	A, B, C, D	[B, D]	2/4 = 1/2
$n = 5$	A, B, C, D, E	[B, E] (or any 2)	2/5

Key Takeaway

When a new element arrives, an old element either survives because the new element is discarded, or it survives because the new element replaces someone else.

These two possibilities together reduce the old element's probability from $\frac{s}{n}$ to $\frac{s}{n+1}$, keeping the sample perfectly uniform!

THE CORE FORMULA

$$\left(1 - \frac{s}{n+1}\right) + \left(\frac{s}{n+1} \times \frac{s-1}{s}\right) = \frac{n}{n+1} \Rightarrow \frac{s}{n} \times \frac{n}{n+1} = \frac{s}{n+1}$$

- $1 - \frac{s}{n+1}$: New element discarded
- $\frac{s}{n+1} \times \frac{s-1}{s}$: New element chosen but does not replace this element
- $\frac{n}{n+1}$: Probability an old element survives
- $\frac{s}{n} \times \frac{n}{n+1} = \frac{s}{n+1}$: Equal probability for all!

Queries over a (long) Sliding Window

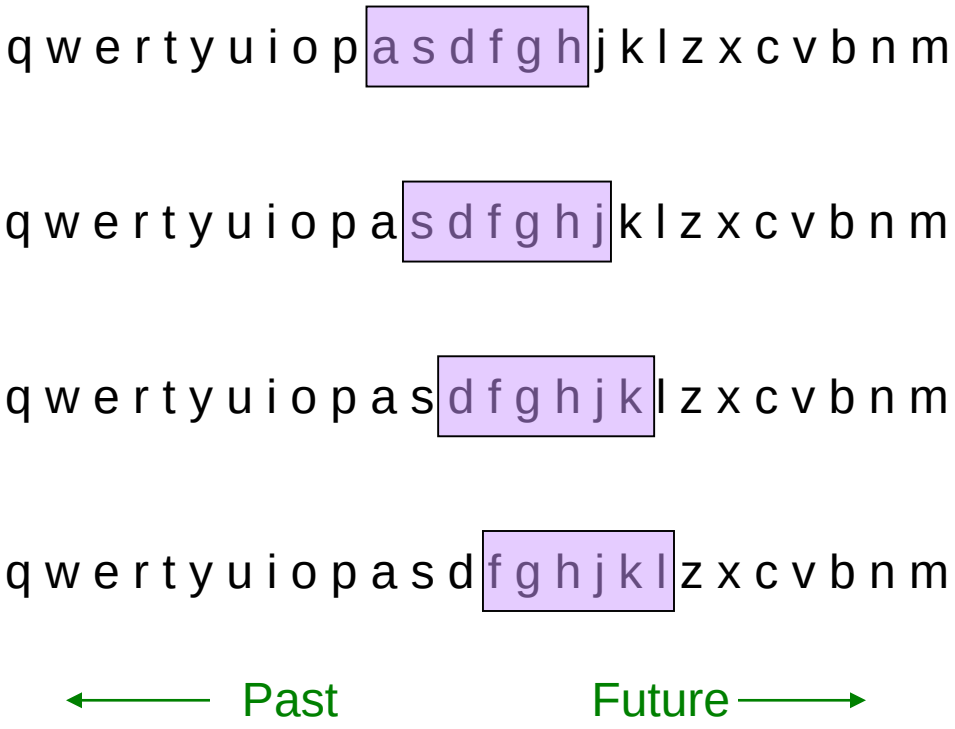
Sliding Windows

- A useful model of stream processing is that queries are about a ***window*** of length ***N*** – the ***N*** most recent elements received

- **Interesting case: *N*** is so large that the data cannot be stored in memory, or even on disk
  - Or, there are so many streams that windows for all cannot be stored
- **Amazon example:**
  - For every product **X** we keep 0/1 stream of whether that product was sold in the **n**-th transaction
  - We want answer queries, how many times have we sold **X** in the last **k** sales

Sliding Window: 1 Stream

- **Sliding window on a single stream:** **N = 6**



SLIDING WINDOWS & COUNTING BITS

Focus only on the Recent, Not the Entire History

1 WHAT IS A SLIDING WINDOW?

A sliding window of length *N* contains the *N* most recent elements of a stream.

Example (*N* = 5)

A	B	C	D	E	F	G	H	I	J

Old (ignored) — Current Window (last 5 elements)

Real-World Examples

Google Search

How many times was "ChatGPT" searched in the last 10 minutes?

Amazon Sales

How many laptops were sold in the last 1000 transactions?

(Laptop sold? Yes = 1, No = 0)

Why useful? Recent data is more important, old data becomes irrelevant, memory is limited.

2 SLIDING WINDOW ON ONE STREAM

- Stream: q w e r t y u i o p a s d f g h j k l z x c v b n m ...

Step 1: q w e r t y u i o p a **a s d f g h** k l z x c v b n m ...

Step 2: q w e r t y u i o p a s **s d f g h j** k l z x c v b n m ...

Step 3: q w e r t y u i o p a s s **d f g h j k** l z x c v b n m ...

Step 4: q w e r t y u i o p a s s d **f g h j k l** z x c v b n m ...

Every time, one old element leaves and one new element enters.

← Past (older) → ← Future (newer) →

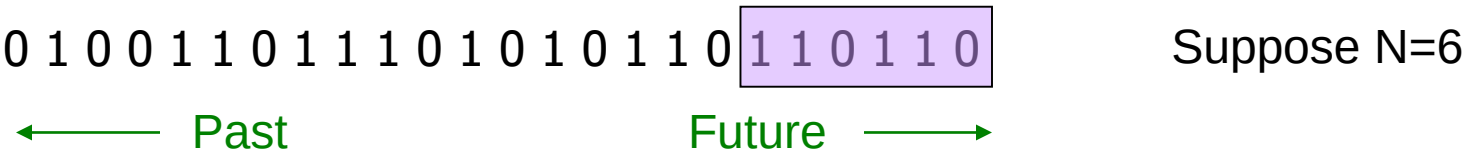
The processor always focuses only on the most recent *N* elements.

Counting Bits (1)

- **Problem:**
  - Given a stream of **0**s and **1**s
  - Be prepared to answer queries of the form **How many 1s are in the last *k* bits?** where $k \leq N$

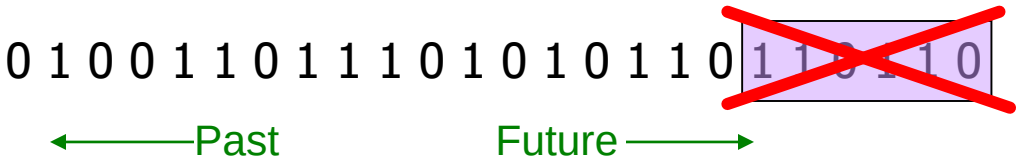
- **Obvious solution:**

  Store the most recent N bits
  - When new bit comes in, discard the $N+1^{st}$ bit



Counting Bits (2)

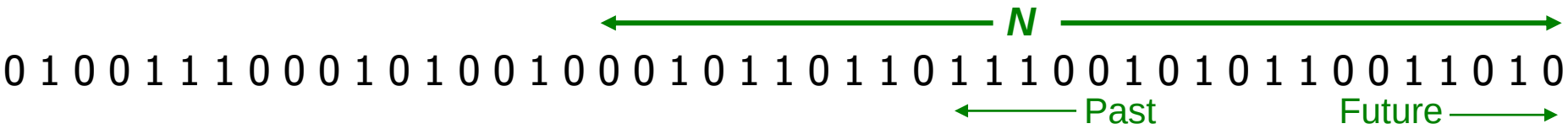
- **You can not get an exact answer without storing the entire window**
- **Real Problem:**
  What if we cannot afford to store *N* bits?
  - **E.g.**, we're processing 1 billion streams and ***N* = 1 billion**



- **But we are happy with an approximate answer**

An attempt: Simple solution

- **Q: How many 1s are in the last *N* bits?**
- A simple solution that does not really solve our problem: **Uniformity assumption**



- **Maintain 2 counters:**
  - ***S***: number of 1s from the beginning of the stream
  - ***Z***: number of 0s from the beginning of the stream
- **How many 1s are in the last N bits?** $N \cdot \frac{S}{S+Z}$
- **But, what if stream is non-uniform?**
  - What if distribution changes over time?

3 COUNTING BITS (1) – THE PROBLEM

Stream of bits (0s and 1s):

0 1 0 0 1 0 1 0 0 1 0 ...

Example (N = 6)

Q: How many 1s are in the last *k* bits?

Current stream:

0 1 0 0 1 0 1 0 1 0

0 1 1 0 1 0

Last 6 bits (current window)

Number of 1s in last 6 bits = 3

Exact Solution: Store the last *N* bits.

- When a new bit arrives:
  - Add the new bit
  - Remove the oldest bit



This gives the exact answer, but storing *N* bits may be impossible if *N* is huge.

4 COUNTING BITS (2) – WHY EXACT SOLUTION IS IMPRACTICAL

- Suppose *N* = 1 billion.
- Storing 1 billion bits for one stream is already huge.
- If we have 100 million streams? Impossible!
- Example: 100 million IoT sensors, each needs last 1 billion bits. We cannot store that much data.



Therefore, we are happy with an APPROXIMATE ANSWER instead of exact.



5 AN ATTEMPT – SIMPLE SOLUTION (BUT FLAWED)

Maintain only two counters from the beginning of the stream.

S: number of 1s seen so far (S)

Z: number of 0s seen so far (Z)

Example Stream:

1	0	1	1	0	0	1

Here, S = 4 (1s), Z = 3 (0s)

Uniformity Assumption:

Assume 1s and 0s are uniformly distributed.

$$\text{Fraction of 1s} = \frac{S}{S+Z}$$



40% 1s 60% 0s

Example: S = 400, Z = 600 → Total = 1000

$$\text{Fraction of 1s} = \frac{400}{1000} = 0.4$$

Suppose $N = 100$

Estimated 1s in last N bits = $100 \times 0.4 = 40$

6 WHY THIS SIMPLE SOLUTION FAILS

Streams are NOT always uniform!

Counter Example:

First half:	0	0	0	0	0	0	0	0
Second half:	1	1	1	1	1	1	1	1

Overall: 50% 1s

Ask: How many 1s in the last 10 bits?

Correct answer = 10

Simple estimation = 5 ✗

Reason: Distribution changes over time (non-uniform stream).
This is called NON-UNIFORM STREAM or CONCEPT DRIFT.



KEY TAKEAWAYS





Sliding Window keeps only the most recent N elements.



As new data arrives, the window slides forward.



Many real-world applications care only about recent data.



Storing last N bits exactly is impractical when N is very large or streams are many.



We need efficient approximate methods (DGIM) to answer queries on windows.

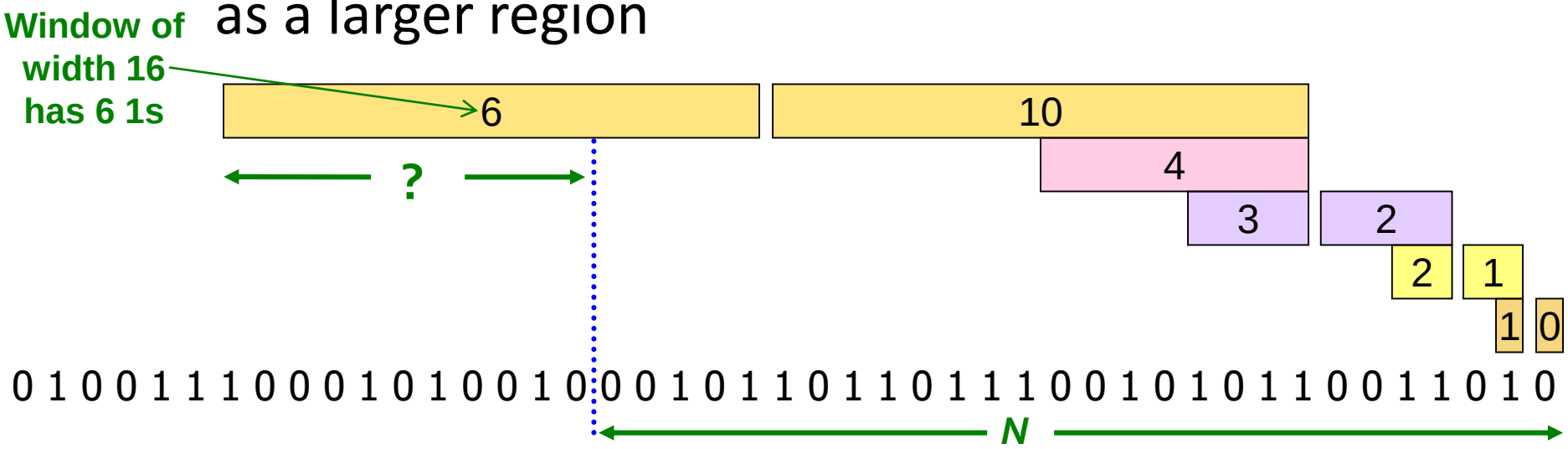
[Datar, Gionis, Indyk, Motwani]

DGIM Method

- **DGIM solution that does not assume uniformity**
- We store $\boldsymbol{O}(\log^2 \boldsymbol{N})$ bits per stream
- **Solution gives approximate answer, never off by more than 50%**
  - Error factor can be reduced to any fraction > 0, with more complicated algorithm and proportionally more stored bits

Idea: Exponential Windows

- **Solution that doesn't (quite) work:**
  - Summarize **exponentially increasing** regions of the stream, looking backward
  - Drop small regions if they begin at the same point as a larger region



We can reconstruct the count of the last ***N*** bits, except we are not sure how many of the last **6 1s** are included in the ***N***

What's Good?

- **Stores only O($\log^2 N$) bits**
  - $O(\log N)$ counts of $\log_2 N$ bits each
- **Easy update as more bits enter**
- Error in count no greater than the number of **1s** in the "**unknown**" area

DGIM ALGORITHM – WHY MEMORY IS ONLY $O(\log^2 N)$ BITS

DGIM does **NOT** store every bit of the stream. It stores only a small set of **"buckets"** that summarize the data.

1. WHAT DOES DGIM STORE?

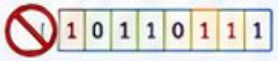
Each bucket stores only **TWO** pieces of information:

- **End Timestamp** (position / time of the last bit in this bucket)
- **Bucket Size** (number of 1s in this bucket, always a power of 2)

Example bucket:

End Timestamp	Size (# of 1s)
1250	8

We do NOT store the individual bits!



2. HOW MANY BUCKETS CAN EXIST?

Bucket sizes in DGIM are powers of 2:

1 | 2 | 4 | 8 | 16 | 32 | 64 | ...

$\log_2 N$ different sizes

DGIM (original) allows **AT MOST TWO** buckets of each size.

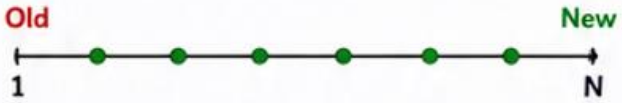
Size	1	2	4	8	16	32	...
Max buckets	≤ 2	≤ 2	≤ 2	≤ 2	≤ 2	≤ 2	...

Therefore, total number of buckets ≤ 2 × (number of sizes) = $O(\log N)$



3. HOW MANY BITS ARE NEEDED PER BUCKET?

(A) BITS FOR END TIMESTAMP



End timestamp can be any number between 1 and N.

To represent numbers up to N, we need $\log_2 N$ bits.

Example: If $N = 1{,}000{,}000$

$\log_2(1{,}000{,}000) \approx 20$ bits

So timestamp $\approx$ 20 bits.



(B) BITS FOR BUCKET SIZE

Bucket size is always a power of 2:

1	2	4	8	16	32	64	...	$\leq N$

Instead of storing the actual size, we store its **EXPONENT.**

Maximum exponent = $\log_2 N$

To store numbers up to $\log_2 N$, we need $\log_2(\log_2 N)$ bits.

Example: If $N = 1{,}000{,}000$

$\log_2 N \approx 20 \rightarrow \log_2(20) \approx 5$ bits

So size info $\approx$ 5 bits.

We store exponent, not the actual size!

e.g., $1024 = 2^{10}$

Store 10 (not 1024)



Total bits per bucket $\approx \log_2 N$ (timestamp) + $\log_2 \log_2 N$ (size)

Since $\log \log N << \log N$, this is $O(\log N)$ bits per bucket.

4. TOTAL MEMORY REQUIRED

Number of buckets $= O(\log N)$  × Bits per bucket $= O(\log N)$ bits  → **Total Memory $= O(\log N) \times O(\log N)$ $= \underline{O(\log^2 N)}$ bits**

5. NUMERICAL EXAMPLE

Suppose window size $N = 1,024$

Possible bucket sizes:

1	2	4	8	16	32	64	128	256	512	1024

Number of sizes $= \log_2 1024 = 10$

Max buckets (2 per size) $= 2 \times 10 = 20$

Bits needed per bucket:

- Timestamp: $\log_2 1024 = 10$ bits
- Size (exponent): $\log_2 10 \approx 4$ bits

Total per bucket ≈ 14 bits



- Total memory ≈ 20 buckets × 14 bits = **280 bits**

(Instead of storing 1024 bits!)

SUMMARY TABLE

Item	DGIM
Number of bucket sizes	$\log_2 N$
Max buckets per size	2 (original DGIM)
Total number of buckets	$O(\log N)$
Bits per bucket	$O(\log N)$
Total memory	$O(\log^2 N)$ bits



KEY TAKEAWAY

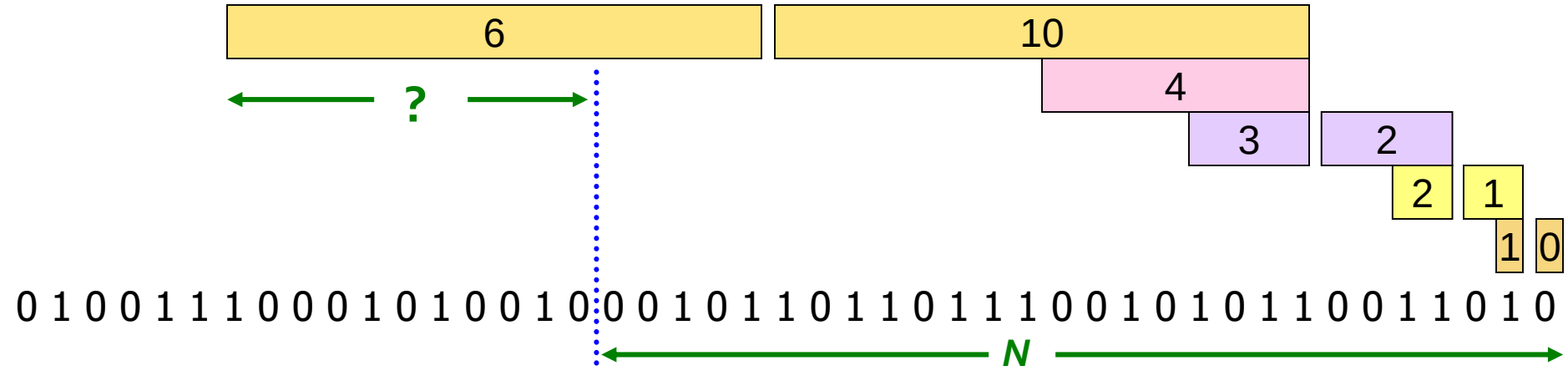
- DGIM stores only bucket summaries, not individual bits.
- Bucket sizes are powers of 2 → only $\log_2 N$ different sizes.
- At most 2 buckets per size → total buckets $= O(\log N)$.
- Each bucket needs $O(\log N)$ bits to store.



Hence, DGIM uses only $O(\log^2 N)$ bits of memory while giving an estimate with bounded error (≤ 50% in original DGIM).

What's Not So Good?

- As long as the **1s** are fairly evenly distributed, the error due to the unknown region is small – **no more than 50%**
- But it could be that all the **1s** are in the unknown area at the end
- In that case, **the error is unbounded!**

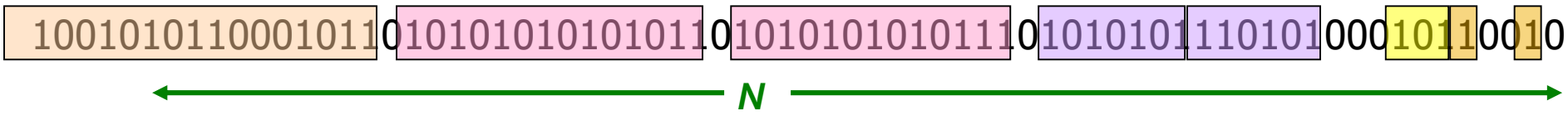


[Datar, Gionis, Indyk, Motwani]

Fixup: DGIM method

- **Idea:** Instead of summarizing fixed-length blocks, summarize blocks with specific number of **1s**:
  - Let the block ***sizes*** (number of **1s**) increase exponentially

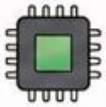
- **When there are few 1s in the window, block sizes stay small, so errors are small**



DGIM METHOD – OVERVIEW & MOTIVATION

Goal: Estimate the number of 1s in the last N bits of a very long (or infinite) stream using very little memory.

1. DGIM METHOD – THE BIG PICTURE



Very small memory!

- DGIM solution that does NOT assume uniformity.
- We store only $O(\log^2 N)$ bits per stream.
- It gives an approximate answer, never off by more than 50%.

Approach	Memory	Answer
Store last N bits	N bits (very large)	Exact
Two counters (total 1s, 0s)	$O(1)$ bits	Wrong when distribution changes
DGIM Method	$O(\log^2 N)$ bits	Approximate (error ≤ 50%)



Example: If N = 1,000,000

- Storing all bits = 1,000,000 bits
- DGIM needs about: $\log_2(10^6) \approx 20 \rightarrow 20^2 = 400$ bits only!

2. WHY DO WE NEED THIS?



Focus only on recent data (latest N bits).



Storing last N bits exactly is impossible when N is huge or many streams.



We need efficient approximate methods.



DGIM provides approximate counts with small memory and fast updates.



Takeaway

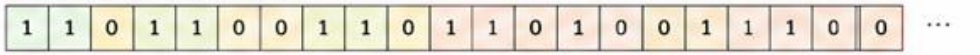
DGIM summarizes the stream using exponentially sized buckets of 1s.
It uses only $O(\log^2 N)$ bits and answers queries with at most 50% error.

DGIM METHOD – HOW IT WORKS & PROPERTIES

1. IDEA – EXPONENTIAL WINDOWS (BUCKETS)

- Instead of storing every bit, summarize the stream using exponentially increasing windows (buckets) looking backward.
- Drop small regions if they begin at the same point as a larger region.

Example Stream (0s and 1s):





DGIM keeps buckets whose sizes increase by powers of 2 (showing the COUNT OF 1s)



Each number in a bucket = number of 1s in that region (window).
We do **NOT** store every bit, only these summaries!

WHAT'S GOOD?

Uses only $O(\log^2 N)$ bits (huge memory saving).

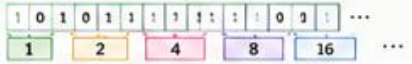
Easy to update as new bits arrive (only a few buckets change).

Error in count comes only from the "unknown" (partial) bucket and is bounded.

3. WHAT'S NOT SO GOOD?

- DGIM gives an approximate answer.
- Error happens because of the oldest (partial) bucket where we don't know exactly how many 1s are in it.

Best Case (1s are spread uniformly)



Error is small. Estimate is very accurate.

Worst Case (all 1s in the oldest bucket)



Error can be as large as 50%.

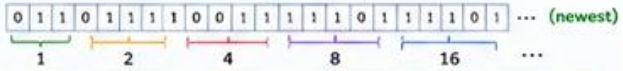
4. FIXUP – THE DGIM METHOD

Old (naïve) idea:
Use fixed-length blocks (e.g., every 8 bits).



Problem: Each block may have very different # of 1s → large error.

DGIM idea (better):
Use buckets based on NUMBER OF 1s, not fixed length.
Bucket sizes (by # of 1s) increase exponentially.



✓ Few 1s → buckets stay small → less uncertainty → more accurate.
✓ Many 1s near present → more small buckets → still accurate!

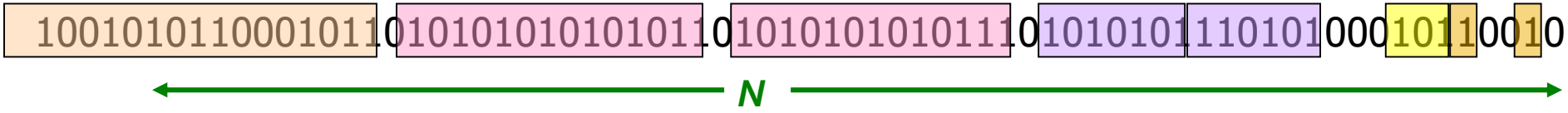
DGIM: Timestamps

- Each bit in the stream has a ***timestamp***, starting **1**, **2**, …
- Record timestamps modulo N (**the window size**), so we can represent any **relevant** timestamp in $O(log_2 N)$ bits

DGIM: Buckets

- A ***bucket*** in the DGIM method is a record consisting of:
  - **(A) The timestamp of its end** [O(log *N*) bits]
  - **(B) The number of 1s between its beginning and end** [O(log log *N*) bits]

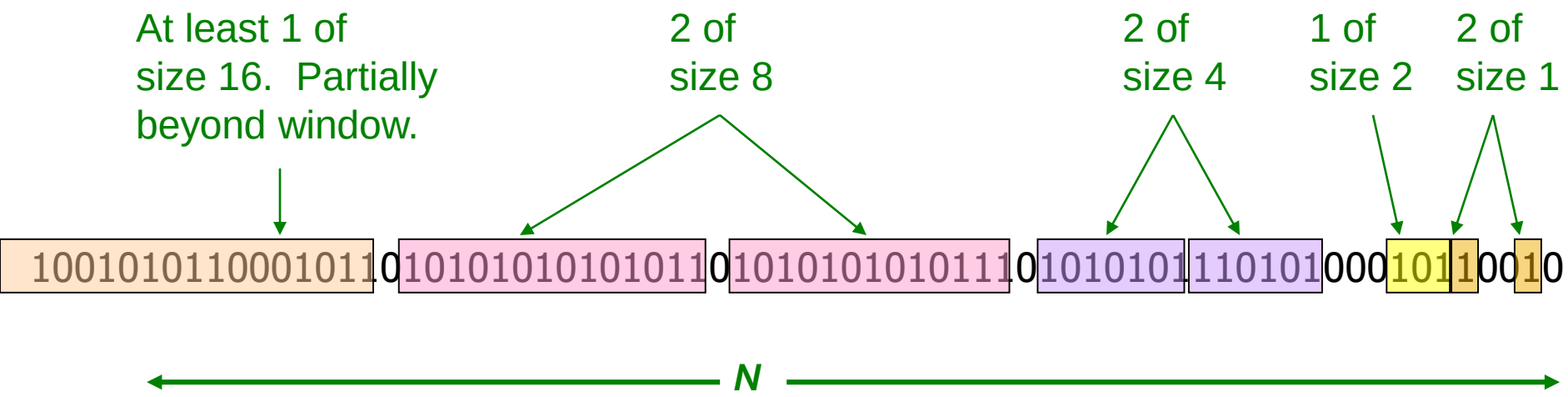
- **Constraint on buckets:**
  Number of **1s** must be a power of **2**
  - That explains the **O(log log *N*)** in **(B) above**



Representing a Stream by Buckets

- Either **one** or **two** buckets with the same **power-of-2 number** of **1s**
- **Buckets do not overlap in timestamps**
- **Buckets are sorted by size**
  - Earlier buckets are not smaller than later buckets
- Buckets disappear when their end-time is **> *N*** time units in the past

Example: Bucketized Stream



Three properties of buckets that are maintained:

- Either **one** or **two** buckets with the same **power-of-2** number of **1s**
- Buckets do not overlap in timestamps
- Buckets are sorted by size

Updating Buckets (1)

- When a new bit comes in, drop the last (oldest) bucket if its end-time is prior to ***N*** time units before the current time
- **2 cases:** Current bit is **0** or **1**
- **If the current bit is 0:**
  no other changes are needed

Updating Buckets (2)

- **If the current bit is 1:**
  - **(1)** Create a new bucket of size **1**, for just this bit
    - **End timestamp = current time**
  - **(2)** If there are now **three buckets of size 1**, **combine the oldest two into a bucket of size 2**
  - **(3)** If there are now **three buckets of size 2**, **combine the oldest two into a bucket of size 4**
  - **(4) And so on …**

DGIM METHOD – BUCKETS & REPRESENTING A STREAM

1 DGIM BUCKETS – THE BASICS

DGIM summarizes a long stream to estimate the number of 1s in the last N bits using very little memory.

A BUCKET stores only TWO pieces of information

A End Timestamp

Time (position) of the last 1 in this bucket.

Range: [0 .. N]

B Number of 1s

Count of 1s in this bucket.
Must always be a power of 2.

Possible sizes: 1, 2, 4, 8, 16, 32, ...

1 | 2 | 4 | 8 | 16 | ...

EXAMPLE

Stream (positions)

1	2	3	4	5	6	7	8	9 ...
1	0	1	1	0	1	0	1	1 ...

→

Bucket stored by DGIM

End Time = 9
Count = 3 $(2^1 + 2^0 = 3)$

WHY ONLY TWO VALUES?

These two values are sufficient to estimate how many 1s are in the last N bits, saving huge memory.

MEMORY BENEFIT

Instead of storing all N bits, DGIM stores only $O(\log^2 N)$ values.

Example: If N = 1,000,000 → $\log_2(10^6) \approx 20$ → 20^2 = 400 values only!

2 REPRESENTING A STREAM BY BUCKETS

DGIM maintains a set of buckets that summarize the stream.

1. At most ONE or TWO buckets of any size (power of 2).
   - ✓ Allowed: 1 | 1 | 2 | 2 | 4 | 4 | 8
   - ✗ Not allowed: three buckets of same size: 1 | 1 | 1 (three of size 1)
2. Buckets DO NOT overlap in timestamps.
   - ✓ Correct (no overlap): [10–15] [16–22] [23–30]
3. Buckets are SORTED by size. Smaller (newer) on the right, larger (older) on the left.
   - Older ⟷ Newer
   - 16 | 8 | 8 | 4 | 2 | 1 | 1
4. Remove buckets whose end time is > N time units in the past.

Current time = T, Window size = N
Discard any bucket with end time < $T - N$

EXAMPLE: BUCKETIZED STREAM

Example Stream (0s and 1s)

1	0	1	1	0	1	1	0	1	1	0	1	1	0	0	...

Older (Past) ⟷ Newer (Future)

Buckets maintained by DGIM

	16	8	8	2	1	1
End Time	23	31	40	49	52	53	54
Size (# of 1s)	16	8	8	4	2	1	1

Properties satisfied:
- ✓ At most 2 buckets of each size
- ✓ No overlap in timestamps
- ✓ Buckets sorted by size

DGIM METHOD – UPDATING BUCKETS

1 UPDATING BUCKETS – OVERVIEW

When a new bit arrives at current time T, first delete any bucket whose end time $< T - N$ (outside the window).



Then handle the new bit.

CASE 1: NEW BIT = 0

We do NOT create any bucket.

No other changes are needed (only deletion of expired buckets if any).

0

CASE 2: NEW BIT = 1

Create/merge buckets using DGIM rules.

1

2 CASE 2 (NEW BIT = 1) – STEPS

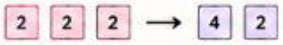
1. Create a new bucket of size 1 for this bit.
   - End timestamp = Current time T.

New bucket
Size = 1
End Time = T

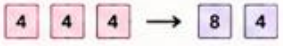
2. If there are NOW THREE buckets of size 1, merge the OLDEST TWO → one bucket of size 2.



3. If there are NOW THREE buckets of size 2, merge the OLDEST TWO → one bucket of size 4.



4. If there are NOW THREE buckets of size 4, merge the OLDEST TWO → one bucket of size 8.

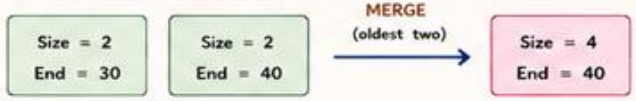


5. Continue this process for sizes 8, 16, 32, ... until every size has at most TWO buckets.

Rule: At most one or two buckets of each size.

3 HOW MERGING WORKS?

Two buckets of same size (older two)



Why?

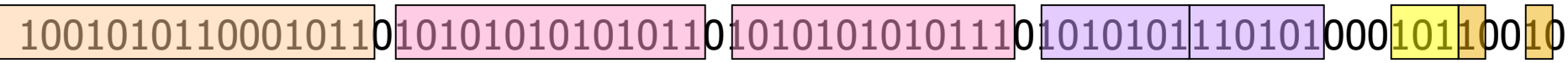
2 ones + 2 ones = 4 ones.
Information is preserved in a compact form.



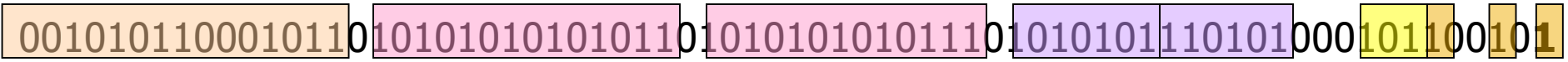
Key Idea: We keep only a small number of buckets while still being able to estimate the number of 1s in the last N bits with bounded error.

Example: Updating Buckets

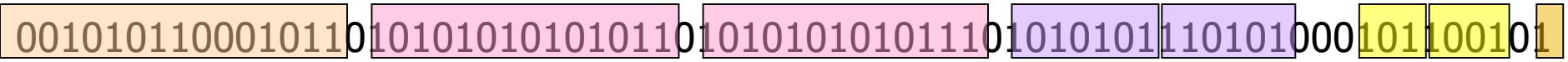
Current state of the stream:



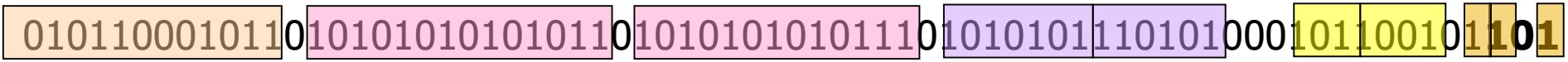
Bit of value 1 arrives



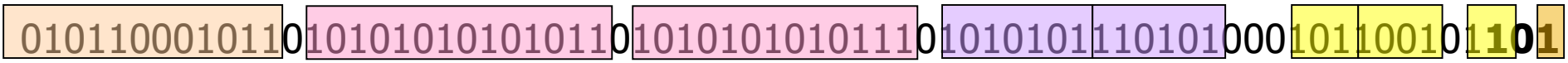
Two orange buckets get merged into a yellow bucket



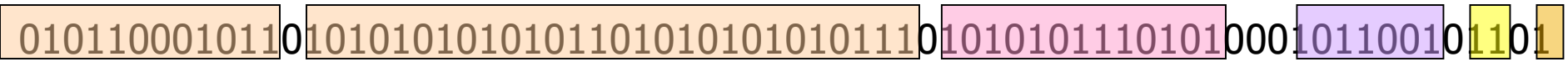
Next bit 1 arrives, new orange bucket is created, then 0 comes, then 1:



Buckets get merged…



State of the buckets after merging



DGIM ALGORITHM – UPDATING BUCKETS

DGIM maintains buckets that summarize the stream. When a new bit arrives, buckets may be created and **merged** to keep the rules satisfied.

BUCKET RULES (RECAP)

- ✓ At most TWO buckets of each size (power of 2).
- ✓ Buckets do NOT overlap in timestamps.
- ✓ Buckets are sorted by size (small → large as we go older).

Bucket size legend (number of 1s)

1 | 2 | 4 | 8 | 16 | ...

1 INITIAL BUCKET STATE

Current state of buckets (from older to newer)

8 | 4 | 4 | 2 | 2 | 1 | 1

Older (Past) ⟷ Newer (Recent)

All DGIM rules satisfied (≤ 2 buckets of each size).

2 STEP 1: NEW BIT = 1 ARRIVES

A new bit 1 arrives at the current time.

New bucket (size 1)

8 | 4 | 4 | 2 | 2 | 1 | 1 | 1

Older (Past) → Newer (Recent)

Three buckets of size 1 → **Not Allowed!**

3 STEP 2: MERGE OLDEST SIZE-1 BUCKETS

Merge the two OLDEST buckets of size 1 into one bucket of size 2.

Merge oldest two size-1 buckets

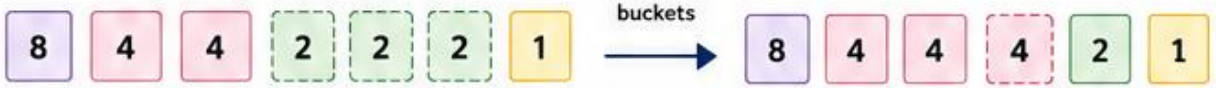
8 | 4 | 4 | 2 | 2 | 2 | 1

Now size-1 is OK. But we have three buckets of size 2 → need to merge again.

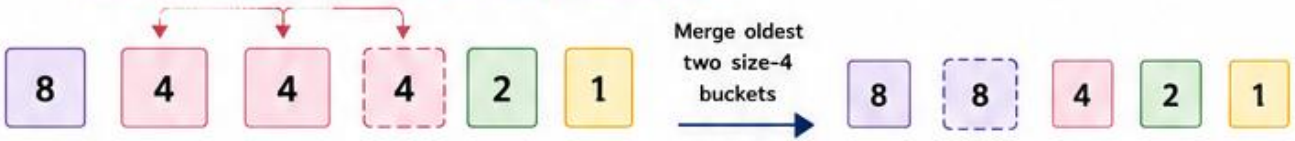
4 STEP 3: CASCADE MERGE

Keep merging while there are THREE buckets of any same size.

First, merge size-2 buckets



Now three buckets of size 4 → merge oldest two



Cascading may continue at higher sizes (4 → 8 → 16 ...) until all rules hold.

5 FINAL BUCKET STATE

After all necessary merges, DGIM rules are satisfied again.



Every bucket size appears at most twice. Ready for future updates!

KEY INSIGHT

1 New bit = 1

Create a new bucket of size 1 (end time = current time).



2 Three equal buckets?

Merge the OLDEST two into one bucket of double size.



3 Repeat merging

This may trigger cascading merges at larger sizes.



4 Why it works

Recent data is kept in small buckets (high accuracy). Older data is summarized in larger buckets (low memory).

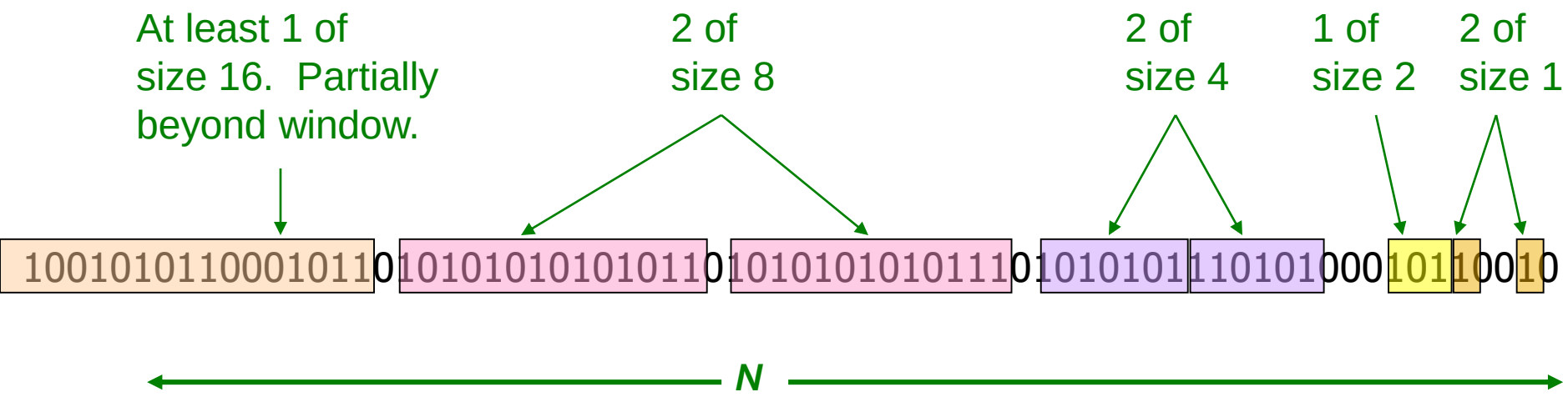
This strategy ensures DGIM uses only $O(\log^2 N)$ space while providing an approximate count of 1s in the last N bits with error at most **50%**.

How to Query?

- **To estimate the number of 1s in the most recent *N* bits:**
  1. **Sum the sizes of all buckets but the last**
     (note "size" means the number of 1s in the bucket)
  2. **Add half the size of the last bucket**

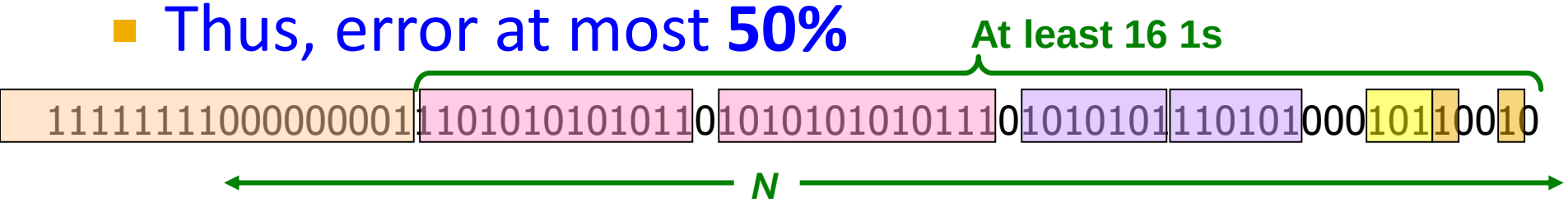
- **Remember:** We do not know how many **1s** of the last bucket are still within the wanted window

Example: Bucketized Stream



Error Bound: Proof

- **Why is error 50%? Let's prove it!**
- Suppose the last bucket has size 2^r
- Then by assuming 2^{r-1} (i.e., half) of its **1s** are still within the window, we make an error of at most 2^{r-1}
- Since there is at least one bucket of each of the sizes less than 2^r, the true sum is at least $1 + 2 + 4 + .. + 2^{r-1} = 2^r - 1$
- Thus, error at most **50%**



DGIM ALGORITHM – HOW TO QUERY?

Goal: Estimate the number of 1s in the most recent N bits using the current set of buckets.

1 HOW TO QUERY?

DGIM QUERY RULE

1. Sum the sizes of ALL buckets except the oldest (leftmost) bucket.
2. Add HALF of the size of the oldest bucket.

Why half?

The oldest bucket may be partially outside the window. We do not know how many 1s from it are inside, so we take half as estimate.

EXAMPLE

Current buckets (older → newer)



Oldest bucket (may be partial)

Newest bucket (fully inside)

ESTIMATION

Step 1: Sum all except oldest

$8 + 8 + 8 + 4 + 4 + 2 + 1 + 1 = 28$

Step 2: Add half of oldest

$16 / 2 = 8$

Estimated number of 1s in last N bits = $28 + 8 = 36$

WINDOW VIEW

Sliding window of size N



The oldest bucket (16) is partially outside the window.

Others are completely inside.

This gives a good approximation with small memory!

2 EXAMPLE: BUCKETIZED STREAM

Bucket sizes (older → newer)



Oldest bucket (partially outside window)

Completely inside window

Suppose we query the last N bits.

Estimation using DGIM query rule:

Sum of all except oldest

$= 8 + 8 + 4 + 4 + 2 + 1 + 1$

$= 28$

Half of oldest bucket

$= 16 / 2 = 8$

Estimated count $= 28 + 8 =$ **36**

Key Points

- We count all buckets exactly **except the oldest.**
- Only the **oldest bucket** is estimated as **half.**
- Error comes only from the **oldest bucket.**

3 WHY IS THE ERROR AT MOST 50%?

Let's prove the error bound.

Let the **oldest bucket** have size 2^r.

- We do not know how many of its 1s are inside the window.
- DGIM counts half of it: 2^{r-1}.

Why is this at most 50% of the true count?

Because there is at least one bucket of every size smaller than 2^r completely inside the window.



Minimum guaranteed number of 1s from these buckets:

$1 + 2 + 4 + \ldots + 2^{r-2} + 2^{r-1} = 2^r - 1$

Example (r = 4)

Oldest bucket size $= 2^4 = 16$

- We count half = 8
- Minimum guaranteed from smaller buckets = $1 + 2 + 4 + 8 = 15$

Relative error $\leq \frac{8}{15} \approx 53\% < 50\%$

Maximum possible error

At most 2^{r-1} (if none or all of this bucket lies outside/inside).

Maximum error $= 2^{r-1}$ → Minimum true count $\geq 2^r - 1$ → Relative error $= \frac{2^{r-1}}{2^r - 1} <$ **50%**

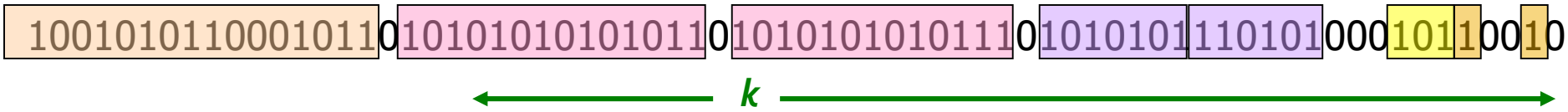
Thus, DGIM guarantees **ERROR AT MOST 50%** while using only $O(\log^2 N)$ space!

Further Reducing the Error

- Instead of maintaining **1** or **2** of each size bucket, we allow either ***r*-1** or ***r*** buckets (***r* > 2**)
  - Except for the largest size buckets; we can have any number between **1** and ***r*** of those
- **Error is at most *O*(1/*r*)**
- By picking ***r*** appropriately, we can tradeoff between number of bits we store and the error

Extensions

- Can we use the same trick to answer queries **How many 1's in the last *k*?** where ***k*** **<** ***N***?
  - **A:** Find earliest bucket **B** that at overlaps with ***k***. Number of **1s** is the **sum of sizes of more recent buckets + ½ size of B**

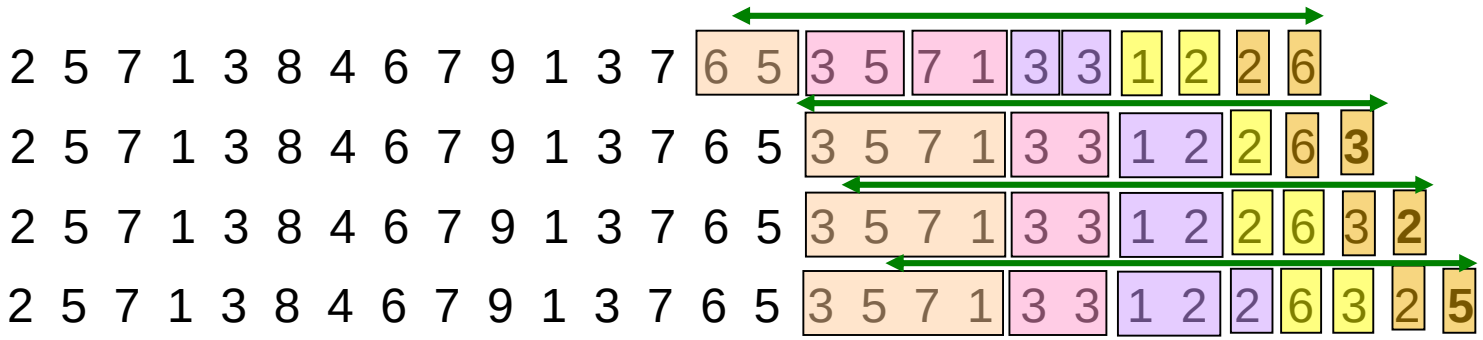


- **Can we handle the case where the stream is not bits, but integers, and we want the sum of the last *k* elements?**

Extensions

- **Stream of positive integers**
- **We want the sum of the last *k* elements**
  - **Amazon:** Avg. price of last **k** sales
- **Solution:**
  - **(1) If you know all have at most *m* bits**
    - Treat ***m*** bits of each integer as a separate stream
    - Use DGIM to count **1s** in each integer
    - The sum is $= \sum_{i=0}^{m-1} c_i 2^i$

    c_i …estimated count for **i-th** bit
  - **(2) Use buckets to keep partial sums**
    - **Sum of elements in size *b* bucket is at most 2^b**



Idea: Sum in each bucket is at most 2^b (unless bucket has only **1** integer)

Bucket sizes:

16	8	4	2	1

DGIM ALGORITHM – ADVANCED EXTENSIONS

1 FURTHER REDUCING THE ERROR



Goal: Decrease the estimation error by storing more buckets per size.

ORIGINAL DGIM ($r = 2$)

Allow only 1 or 2 buckets of each size.

Example bucket sizes:



GENERALIZED DGIM (parameter $r > 2$)

Allow either $(r-1)$ or r buckets of each size.

Example with $r = 4$ → allow 3 or 4 buckets per size





Why error reduces?

- More buckets → more information stored → smaller uncertain part.
- Uncertainty (at most half of one bucket) becomes smaller relative to the total.



Memory = $O(\log^2 N)$



Max Error ≤ 50%

ERROR vs. MEMORY TRADE-OFF

Parameter r	2 (Original)	4	8	10	r (General)
Buckets per size	1 or 2	3 or 4	7 or 8	9 or 10	$r-1$ or r
Max Error (approx.)	≤ 50%	≤ 25%	≤ 12.5%	≤ 10%	$O(1/r)$
Memory Usage	Lowest	Higher	Higher	Even Higher	Increases with r



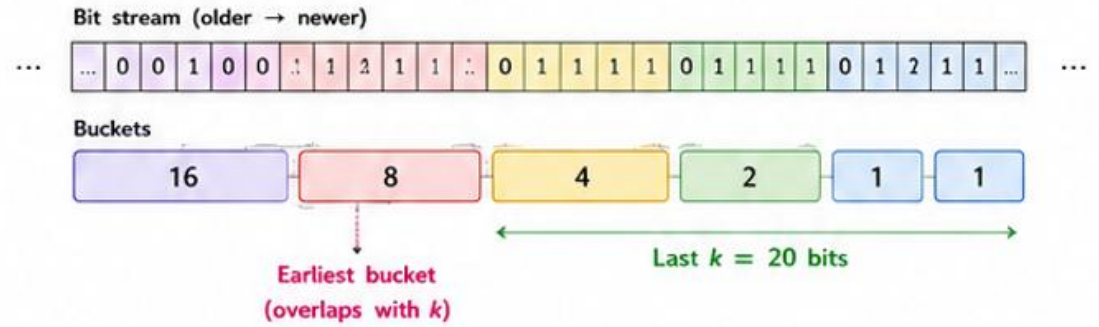
Trade-off:

Increase r to get better accuracy, but memory usage also increases.

2 EXTENSION 1: ANSWERING QUERIES FOR LAST k BITS ($k < N$)

We maintain buckets for the last N bits.

How many 1's in the last k bits ($k < N$)?



QUERY RULE

Find the earliest (oldest) bucket B that overlaps with the k-bit window.

Answer = (sum of sizes of all buckets newer than B) + $1/2 \times$ size(B)

Example

Buckets (older → newer):

16	8	4	2	1	1

Query: last k = 20 bits

The window of 20 bits starts inside bucket 8.

Older Newer →

Estimation

Buckets newer than B: $4 + 2 + 1 + 1 = 8$

Half of bucket B (size 8): $8/2 = 4$

Estimated # of 1's in last 20 bits = $8 + 4 =$ **12**

Key Points

- All newer buckets are fully inside the k-bit window.
- Only bucket B is partial → we count half of it.
- Same idea as DGIM query for window size N.

3 EXTENSION 2: STREAM OF INTEGERS

Problem

Stream contains positive integers.

We want the sum of the last k elements.

Example (e.g., Amazon sales prices)

Stream: 2 5 7 1 3 8 4 6 7 7 9 1 3 7 6 5 ...

SOLUTION 1: BIT-BY-BIT APPROACH

1. Assume each integer uses at most m bits.
2. Treat each bit position as a separate bit stream.
3. Run DGIM on each bit position i ($0 \le i < m$) to estimate c_i = # of 1's in i-th bit.
4. Sum of last k elements = $\sum_{i=0}^{m-1} c_i \cdot 2^i$

Integer (in binary)	Bit 3 (2^3)	Bit 2 (2^2)	Bit 1 (2^1)	Bit 0 (2^0)
6	0	1	0	0
5	0	1	0	1
3	0	1	1	1
...	DGIM	DGIM	DGIM	DGIM
	c_3	c_2	c_1	c_0

$$\text{Sum} = \sum_{i=0}^{m-1} c_i 2^i$$

SOLUTION 2: STORE PARTIAL SUMS IN BUCKETS (MORE EFFICIENT)

Instead of counting bits, store the *SUM* of values in each bucket.

Each bucket keeps two things:

(1) size = number of elements (a power of 2)

(2) sum = sum of those elements



Size (count)

Sum of values

Example

Stream (older → newer)

2 5 7 3 8 4 4 6 7 9 3 7 6 5

Grouped into buckets

16	8	4	2	1	1
(sum = 60)	(sum = 34)	(sum = 16)	(sum = 10)	(sum = 7)	(sum = 5)

Query: sum of last k elements

- Find earliest overlapping bucket B.
- Answer = (sum of newer buckets) + $\frac{1}{2}$ × sum(B).

Idea

Recent buckets are small → accurate.

Older buckets are large → compact.

Same as DGIM, but we store partial sums instead of counts of 1's.

SUMMARY

1 Reduce Error

Allow $(r-1)$ or r buckets per size ($r > 2$).

Error = $O(1/r)$ (smaller r → larger memory).

$r = 2$: 1 1

$r = 4$: 1 1 1 1

2 Last k Bits Query

Find earliest overlapping bucket.

Answer = sum(newer buckets) + $\frac{1}{2}$ × size(overlapping bucket).

16	8	4	2	1	1

k

3 Integer Streams



Either use DGIM on each bit separately or store partial sums in buckets to estimate the sum of the last k elements.

Takeaway

DGIM is flexible!

We can trade memory for accuracy, answer any recent-window query, and handle streams of integers.

Summary

- **Sampling a fixed proportion of a stream**
  - Sample size grows as the stream grows
- **Sampling a fixed-size sample**
  - Reservoir sampling
- **Counting the number of 1s in the last N elements**
  - Exponentially increasing windows
  - Extensions:
    - Number of 1s in any last k ($k < N$) elements
    - Sums of integers in the last N elements